Document Summary

This note covers the foundational elements of deep learning, starting with the structure of neural networks and the role of perceptrons as basic units. It explores activation functions for introducing non-linearity, optimization techniques like gradient descent and ADAM for training models, and backpropagation for efficient gradient computation. Additionally, it discusses loss functions, evaluation metrics, and strategies to handle underfitting and overfitting in multi-layer perceptron models.

Deep Learning Fundamentals: Perceptrons, Activation Functions, Optimization, and Training

Introduction to Deep Learning

Deep Learning involves **neural networks** Neural Networks composed of interconnected nodes, often called `neurons`, that process input data to produce meaningful outputs. These networks are organized into distinct layers, which work together to transform the data step by step.

To break this down clearly:

- **Input layer**: This is the first layer where neurons directly receive the raw input data, such as pixel values from an image or features from a dataset.
- **Hidden layers**: These are the intermediate layers where neurons perform computations on the data passed from the input layer. A network is considered "deep" if it has three or more hidden layers, allowing for increasingly complex feature extraction.
- **Output layer**: This final layer generates the network's prediction or output, such as a classification label or a numerical value.

Neurons in different layers connect through links, known as `synapses`, which transmit information forward from one layer to the next. Deep Neural Networks (DNNs) can adopt various architectures, including Feed-Forward (data flows in one direction), Recurrent (handles sequences with loops), Graph (for interconnected data like social networks), and Convolutional (specialized for grid-like data such as images). A crucial aspect of building effective DNNs is **model engineering**, which begins with understanding basic components like the single neuron and builds up to the **perceptron** Perceptron.

Perceptron

The perceptron is the simplest form of a neural network unit, mimicking a biological neuron by computing a weighted sum of inputs and applying an activation function.

The perceptron, introduced in 1958, was originally conceived as a probabilistic model mimicking how the brain stores and organizes information. For instance, a simple example of a DNN might consist of 4 hidden layers, with specific numbers of nodes in the input, hidden, and output layers. This structure takes input data and produces an output vector representing predictions.

The Perceptron

The **perceptron** serves as the simplest building block of a neural network, essentially a single neuron unit. It takes input features, assigns weights to them to reflect their importance, computes a weighted sum of these inputs, and then applies an activation function to determine the final output.

The mathematical formulation of a perceptron is as follows:

Perceptron Output

$$y = f\left(w_1 x_1 + w_2 x_2 + \cdots + w_n x_n\right)$$

Here:

- $x = (x_1, x_2, \ldots, x_n)$ represents the input features of a data sample.
- $w = (w_1, w_2, \ldots, w_n)$ are the corresponding weights that scale each input.

For a more detailed and complete expression, including a bias term:

Perceptron with Bias

$$y = f\left(\sum_{i=0}^{n} w_i x_i\right) = f(w^T x)$$

In this setup, $x_0 = 1$ is a constant bias input, which helps incorporate an offset without needing an extra input feature.

To clarify the components:

- $x = (x_1, x_2, \ldots, x_n)$: The features of the input sample.
- y: The output produced by the perceptron.
- $f(\cdot)$: A non-linear **activation function** Activation Functions that introduces non-linearity and decides whether the neuron "fires" (activates).
- w_i: The weights for each input, learned during training.
- w_0: The bias weight, which shifts the input to the activation function and allows the perceptron to represent any linear function by adjusting the decision boundary.

Importantly, all the weights w are learned from the training data through an optimization process.

Note: Apart from the activation function $f(\cdot)$, the perceptron closely resembles a linear regression model. If the activation is set to the sigmoid function $f(\cdot) = \sigma(\cdot)$, it becomes analogous to logistic regression, which is used for binary classification.

Perceptron in a 2D Scenario with Linear Activation

Consider a simple 2D input case where $x = (x_1, x_2)$ and the weight vector is $w = (w_0, w_1, w_2)$, including the bias. The output simplifies to:

> **2D Linear Perceptron**
>
> $$y = w_1 x_1 + w_2 x_2 + w_0$$

In this linear activation scenario, where $f(x) = x$ (no non-linearity), the perceptron represents a family of linear decision boundaries. The specific values of w_0, w_1, w_2 define which particular linear function the perceptron learns, effectively drawing a straight line to separate data points.

> **Gender Prediction with Height and Weight**
>
> *Suppose we use height (x_1) and weight (x_2) to predict a student's gender (male or female). The perceptron could learn weights such that $y > 0$ indicates "male" and $y \leq 0$ indicates "female," creating a linear separator in the height-weight plane.*

To illustrate this computationally, here's a simple Python example implementing a 2D perceptron with linear activation:

```
import numpy as np

def perceptron_linear(x1, x2, w0, w1, w2):
    """
```

```
    Computes the linear output of a 2D perceptron.
    Args:
        x1, x2: Input features (height, weight).
        w0, w1, w2: Bias and weights.
    Returns:
        y: Linear output.
    """
    x = np.array([1, x1, x2])  # Include bias term (x0 = 1)
    w = np.array([w0, w1, w2])
    y = np.dot(w, x)  # w^T x
    return y

# Numerical example: Height=170 cm, Weight=70 kg, weights w0=-100, w1=0
result = perceptron_linear(170, 70, -100, 0.5, 0.3)
print(f"Output y: {result}")  # Example output: 45.0 (positive, e.g., p
```

This code demonstrates how the weighted sum is calculated, providing a concrete numerical example where specific values yield $y = 45.0$.

Activation Functions

Activation functions are essential components that impose specific properties on the neuron's output—for instance, bounding it between 0 and 1 like the sigmoid function. They introduce non-linearities into the model, which are crucial for learning complex patterns. Additionally, these functions determine whether a neuron activates (e.g., ReLU sets negative inputs to 0, effectively "turning off" the neuron) and help with optimization by promoting faster convergence and encouraging sparsity in the network (where many neurons remain inactive).

Common activation functions include ReLU (Rectified Linear Unit), Sigmoid, Leaky ReLU, Tanh (Hyperbolic Tangent), Softmax, Linear, and GeLU (Gaussian Error Linear Unit). However, some functions, such as Sigmoid and Tanh, can lead to vanishing gradients in very deep networks, where gradients become extremely small during backpropagation, hindering learning in earlier layers.

For a quick comparison, here's a table summarizing key activation functions:

Activation Function	Formula	Range	Key Properties	Use Case
ReLU	$f(x) = \max(0, x)$	$[0, \infty)$	Simple, fast; avoids vanishing gradients	Hidden layers in deep nets

Activation Function	Formula	Range	Key Properties	Use Case
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	$(0, 1)$	Smooth, probabilistic output	Binary classification outputs
Tanh	$f(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$	$(-1, 1)$	Zero-centered; similar to sigmoid but bounded	Hidden layers (older models)
Softmax	$P_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$	$(0, 1)$, sums to 1	Converts logits to probabilities	Multi-class output layers
Linear	$f(x) = x$	$(-\infty, \infty)$	No non-linearity	Regression outputs

Example: Sigmoid Function

In binary classification tasks, the goal is to separate positive and negative samples using a perceptron. For an input $x \in \mathbb{R}^2$, the perceptron computes a raw score, and we interpret it as the probability $p(+|x)$ of the positive class. Since it's binary, $p(+|x) = 1 - p(-|x)$. To ensure this probability lies in [0, 1], we apply the **sigmoid** activation, which maps any real number $\mathbb{R}$ to the interval [0,1]:

> **Sigmoid Activation**
>
> $$\sigma(y) = \frac{1}{1 + e^{-y}}$$

This function "squashes" the perceptron's unbounded output $y \in \mathbb{R}$ into a bounded probability between 0 and 1. For a numerical example, if $y = 2$ (positive logit), then $\sigma(2) \approx 0.88$ (high probability of positive class); if $y = -2, \sigma(-2) \approx 0.12$ (low probability).

Here's a Python snippet to compute the sigmoid:

```
import numpy as np

def sigmoid(y):
    """
    Sigmoid activation function.
    Args:
        y: Input logit (real number).
```

```
    Returns:
        Probability in [0, 1].
    """
    return 1 / (1 + np.exp(-y))

# Numerical example
y_positive = 2
prob_positive = sigmoid(y_positive)
print(f"Sigmoid({y_positive}) = {prob_positive:.2f}")  # Output: 0.88

y_negative = -2
prob_negative = sigmoid(y_negative)
print(f"Sigmoid({y_negative}) = {prob_negative:.2f}")  # Output: 0.12
```

Towards Bigger Networks: Stacking Perceptrons

To build more powerful models, we stack multiple perceptrons to form fully connected (linear) layers. For a network with two output neurons, the computation proceeds as follows: the first layer produces an intermediate output y_1, which becomes the input to the second layer producing y_2.

Specifically:

> **Two-Layer Composition**
>
> $$y_1 = f(w^T x)$$
>
> $$y_2 = f(q^T y_1)$$
>
> *When composed, this results in:*
>
> $$y = y_2 = f\left(q_0 + q^T f(W^T x)\right)$$

In a simplified vector form without explicit bias for brevity, it can be seen as $y = f(Wx)$, where W combines the weights. The "number of parameters" refers to the total count of weights and biases in the model. For example, a small two-layer network might have 6 parameters. In contrast, modern large models like Mistral 7B contain 7 billion parameters, yet are considered "small" by today's standards due to the scale of computation available.

Adding Layers and Non-Linearities

As we add more layers, non-linear activation functions become critical to prevent the model from simplifying into something trivial. Consider a three-layer setup:

> **Three-Layer Network**
>
> $$z = f\left(s^T f(W^T x)\right)$$

If there were no non-linearity (i.e., $f(x) = x$), the entire expression would collapse to a single linear transformation: $z = s^T W^T x$. This means multiple layers would behave no differently than one linear layer, limiting the model's ability to capture complex, non-linear patterns in data.

Non-linear activations, such as ReLU ($f(x) = \max(0, x)$), ensure that each layer can learn distinct transformations, allowing the network to model intricate functions like curves or decision boundaries with multiple segments.

Introduce Non-Linearities in the Model

To emphasize why non-linearities are indispensable, let's examine what happens without them. Suppose $f(x) = x$ (purely linear):

1. The output z can be rewritten using $W' = Ws$, achieving the same result with a single matrix multiplication.
2. The second layer becomes redundant, as no additional expressive power is gained.
3. The entire model reduces to a linear function, incapable of fitting non-linear data.

In contrast, non-linearities like ReLU prevent this collapse, enabling each layer to contribute unique, non-linear transformations that build hierarchical representations of the data.

Multi-Layer Perceptron Models

Multi-Layer Perceptrons (MLPs) are constructed by stacking multiple layers of perceptrons, each separated by non-linear activation functions. This design avoids the linear collapse issue discussed earlier. Without non-linearities, even deep stacks of layers would equate to a single linear transformation, severely limiting the model's capacity.

A foundational result in neural networks is the **Universal Approximation Theorem**, which states that for any continuous function g defined on a compact subset of $\mathbb{R}^n$ and any error tolerance $\epsilon > 0$, there exists a single-hidden-layer feedforward network with a finite number of neurons that can approximate g within ϵ.

- Citation: Cybenko, George. "Approximation by superpositions of a sigmoidal function." Mathematics of control, signals and systems 2.4 (1989): 303-314. Hornik, Kurt, Maxwell

Stinchcombe, and Halbert White. "Multilayer feedforward networks are universal approximators." Neural networks 2, no. 5 (1989): 359-366.

In theory, a single hidden layer suffices for universal approximation, but the required number of neurons and weights is often impractical and unknown in advance. In practice, deeper and narrower networks are preferred because additional layers allow the model to learn hierarchical, abstract representations of the data—starting from simple edges in images to complex objects—which improves generalization to unseen data.

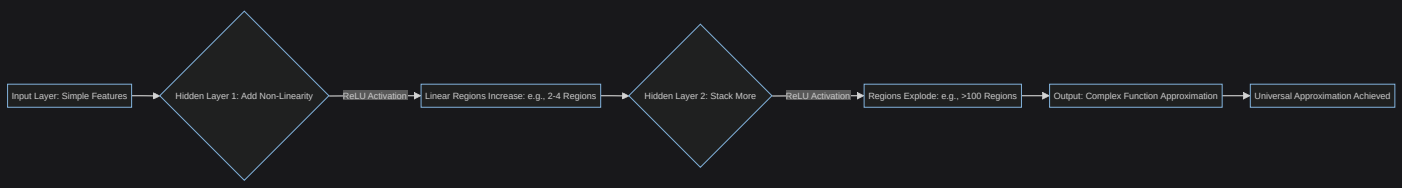
> **Approximating Linear Regions**
>
> *Consider approximating a curve divided into 20 linear regions using a linear model combined with 19 linear neurons. For an input x, one setup might use 4 neurons in the hidden layer; another uses 19. Alternatively, with 1 input x, 4 neurons, or 9 neurons can create more regions. This demonstrates how depth and width increase the number of linear regions the model can fit. (Reference: Simon J.D. Prince "Understanding Deep Learning", November 2024)*

Network Complexity

The complexity of a network, in terms of the number of linear regions it can model, grows significantly with deeper architectures. For a network with 1 linear hidden layer:

- As the number of hidden neurons D_i (or input features) increases, the number of regions explodes. For example, with 500 neurons and $D_i = 100$ features, the model can create over 10×10^7 regions.
- Relating regions to the total number of parameters: A network with 500 hidden neurons and 100 input features has 51,001 parameters (including biases). This is modest compared to modern models like Mistral 7B, which have billions of parameters but enable vastly more complex approximations.

To visualize the growth in expressiveness, consider this Mermaid flowchart showing how stacking layers increases linear regions (using a simple process flow):



Activation Functions for Classification Models

Activation functions play a key role in shaping the output to match the problem type. In classification models, the layers before the final activation produce unnormalized scores called **logits**. These logits are then passed through an output activation to convert them into interpretable probabilities that sum to 1 and lie in [0,1].

The typical structure is: Input → ... → Linear head (final perceptron layer) → Output activation → Probabilities (from logits).

Binary Classification

For binary classification, the model predicts the probability $P(pos|x)$ of the positive class given input x. The logit is $z = model(x)$, and we apply the sigmoid activation:

> **Binary Sigmoid**
>
> $$\sigma(z) = \frac{1}{1 + e^{-z}}$$

This ensures $P(pos|x) \in [0, 1]$, and the negative class probability is $P(neg|x) = 1 - P(pos|x)$, so they sum to 1.

Multi-Class Classification

In multi-class problems, the output y_i belongs to one of $c_1, \ldots, c_n$ classes. The model produces logits $z = (z_1, \ldots, z_n) = model(x)$ from n output perceptrons. To obtain probabilities, apply the **softmax function**:

> **Softmax Function**
>
> $$P(c_i|x) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

The softmax uses all logits to compute relative probabilities: each $P(c_i|x) \in [0, 1]$, and the probabilities sum to 1 across classes.

> **Image Classification with Softmax**
>
> *For classifying images as Dog, Cat, Bird, or Turtle (4 classes), a model with 20 parameters (15 weights + 5 biases, assuming a small input) might output logits like [2.1, 0.5, -1.2, 3.0]. After softmax: approximately [0.35, 0.06, 0.01, 0.58], predicting "Turtle" with highest probability.*

Activation Functions for Regression Models

In regression tasks, the goal is to predict continuous real-valued outputs, so no specific bounds or probabilistic properties are enforced on the output. The most suitable output activation is the

identity function $f(x) = x$, also known as linear activation. This is sensible here because it allows the model to output any real number without distortion.

For a quick numerical illustration: If the linear head computes $z = 5.7$, then $f(z) = 5.7$, directly predicting a value like a house price.

Defining Network Parameters

The weights and biases, collectively denoted as ϕ, start as unknown values and must be tuned using the training data. Training involves iterating over the dataset in passes called **epochs**, where each epoch includes a forward pass to compute predictions. Errors (differences between predicted and true outputs) are then propagated backward via **back-propagation** Backpropagation to update ϕ.

Processing the full dataset at once poses challenges in terms of memory and computation time, especially for large datasets. A practical solution is to use **mini-batches**, which are random subsets of the data sampled without replacement. This approach not only scales better but also improves generalization by introducing variability in updates.

The **learning rate** α controls the size of the update step: it scales the direction of the gradient to balance exploration and precision.

The **objective** or **loss** function $\mathcal{L}(\phi)$ quantifies how well the model performs and is minimized during training. For example, in linear regression, the Mean Squared Error (MSE) is commonly used:

> **Mean Squared Error**
>
> $$\mathcal{L}(\phi) = MSE(\phi) = \frac{1}{N}\sum(y_i - \phi^T x_i)^2$$

Here, $\mathcal{L}$ is minimized with respect to ϕ, while the inputs x_i and true outputs y_i are fixed from the dataset.

Linear Regression

For linear regression, there is a closed-form solution to find the optimal weights by setting the derivative to zero:

> **Optimal Weights for MSE**

$$\frac{\partial \mathcal{L}(\phi)}{\partial \phi} = \frac{\partial MSE(\phi)}{\partial \phi} = 0$$

Since MSE is quadratic in ϕ, this equation can be solved analytically. Alternatively, one could evaluate $\mathcal{L}(\phi)$ for various ϕ and select the one yielding the lowest loss, though this provides less intuitive insight into the optimization process.

More Complex Losses/Models

For non-linear models or more complex losses, no closed-form solution exists. Instead, we evaluate $\mathcal{L}(\phi)$ repeatedly and update ϕ iteratively toward a local minimum. Training starts from random initial values for ϕ and employs an optimization algorithm to guide the updates.

Optimization Algorithms: Gradient Descent (GD)

Gradient Descent (GD) is a foundational optimization method that uses the entire dataset to compute the loss gradient. It updates the parameters incrementally as follows:

Gradient Descent Update

$$\phi_{t+1} := \phi_t - \alpha \nabla_\phi \mathcal{L}(\phi_t)$$

In one dimension, this simplifies to:

1D Gradient Descent

$$\phi_{t+1} := \phi_t - \alpha \frac{\partial \mathcal{L}(\phi_t)}{\partial \phi}$$

The learning rate α determines the step size—too large, and updates overshoot; too small, and convergence is slow. In convex loss landscapes, GD reliably finds the global minimum; however, in non-convex cases (common in deep learning), it may get stuck in local minima.

Toy Example: GD with Simple Linear Regression $y = x\phi_1 + \phi_0$

Consider a toy dataset with $I = 12$ points $\{x_i, y_i\}$. Starting from random initial values near 0, GD iteratively moves "downhill" toward the optimal fit around 1. Multiple epochs are run until convergence (reaching a local minimum or a maximum number of epochs). A heatmap visualization might show the loss landscape with the optimization path: brighter areas indicate higher loss, and after 4 iterations, the path nears the minimum. The fitted line evolves from

light (early epochs) to dark green (best fit). This example highlights GD's behavior in a convex setting. (Reference: Simon J.D. Prince, "Understanding Deep Learning", November 2024)

GD Limitations

GD has notable drawbacks: it is sensitive to initialization (different starting points can lead to different solutions), and in non-convex landscapes, it risks in local minima. Additionally, computing gradients over the full dataset is computationally expensive.

To address these, advanced variants like Stochastic Gradient Descent (SGD), Momentum, and ADAM are used. Convex problems allow convergence to the global minimum from most initializations, but non-convex losses—typical in deep learning—pose ongoing challenges.

Stochastic Gradient Descent (SGD)

While GD relies on the full dataset and is sensitive to initialization, SGD approximates the gradient using a random mini-batch at each step, introducing beneficial noise.

In one dimension:

SGD Update

$$\phi_{t+1} := \phi_t - \alpha \sum_{i \in B_t} \frac{\partial \mathcal{L}_i(\phi_t)}{\partial \phi}$$

Here, B_t is the mini-batch at step t (sampled randomly without replacement within an epoch, ensuring each data point contributes equally once per epoch).

This noisy approximation averages to a downhill direction but can include uphill steps or jumps across valleys, aiding escape from poor local minima.

SGD in Non-Convex Landscape

In a non-convex landscape with 3 possible initializations, standard GD reaches the global minimum in only 1/3 cases, while SGD with the same initializations succeeds in both tested cases, converging faster overall. (Reference: Simon J.D. Prince, "Understanding Deep Learning", November 2024)

Stochastic Properties 1/2

SGD's noise and mini-batch sampling confer several advantages:

1. Noise helps average out erratic updates after an epoch, leading to more sensible parameter adjustments.
2. Sampling without replacement ensures every data point contributes equally within an epoch.
3. It converges faster than full-batch GD due to frequent updates.
4. It requires less computational memory, as only the mini-batch needs to fit in memory.
5. The variability allows escaping local minima more effectively.
6. Larger batches reduce saddle-point issues by averaging gradients more smoothly.
7. SGD often generalizes better to unseen data compared to full-batch methods.

Stochastic Properties 2/2

Unlike GD, SGD does not converge in the traditional sense to a precise minimum; instead, parameters stabilize near the minimum as gradients become small. To manage this, a **learning rate schedule** is used: start with a high α and decrease it by a factor every N epochs.

- Early epochs: High α encourages exploration and valley jumps.
- Later epochs: Lower α enables fine-tuning around the minimum.

Adding Momentum

The variability in mini-batch gradients can cause oscillations, especially in narrow valleys. **Momentum** mitigates this by incorporating a "memory" of past gradients:

1. Compute the momentum term:

> **Momentum Term**
>
> $$m_{t+1} := \beta m_t + (1-\beta) \sum_{i \in B_t} \frac{\partial \mathcal{L}_i(\phi_t)}{\partial \phi}$$

- m accumulates velocity in the update direction.
- $\beta \in [0, 1)$ is the momentum coefficient, controlling how much past information is retained (e.g., $\beta = 0.9$ smooths aggressively).

2. Update parameters using this momentum:

> **Momentum Update**
>
> $$\phi_{t+1} := \phi_t - \alpha m_{t+1}$$

This results in a smoother trajectory, reducing oscillations and accelerating progress through flat regions. (Reference: Simon J.D. Prince, “Understanding Deep Learning”, November 2024)

Fixed Learning Rate Limitations

A fixed learning rate α struggles with varying gradient magnitudes:

- In regions with large gradients, a fixed α causes over-adjustments, leading to instability.
- In regions with small gradients, it results in under-exploration, slowing convergence.

For illustration:

- With a small α: Updates are fast in steep directions (e.g., for ϕ_1) but slow in shallow ones (e.g., for ϕ_2).
- With a large α: The path becomes unstable, with large, erratic changes. (Reference: Simon J.D. Prince, “Understanding Deep Learning”, November 2024)

Gradient Normalization

To handle gradient scale differences, normalization techniques adjust updates to have consistent magnitude while preserving direction. For full-batch GD (adaptable to mini-batches):

1. Compute the first moment (mean gradient): $m_{t+1} := \frac{\partial \mathcal{L}(\phi_t)}{\partial \phi}$
2. Compute the second moment (variance proxy): $v_{t+1} := \left(\frac{\partial \mathcal{L}(\phi_t)}{\partial \phi}\right)^2$
3. Normalize the update:

> **Normalized Update**
>
> $$\phi_{t+1} := \phi_t - \alpha \frac{m_{t+1}}{\sqrt{v_{t+1}} + \epsilon}$$

- ϵ is a small constant (e.g., 10^{-8}) to prevent division by zero.

This ensures updates have a fixed step size in terms of direction, curbing the impact of high-magnitude gradients while treating all directions equally. (Reference: Simon J.D. Prince, “Understanding Deep Learning”, November 2024)

Adaptive Momentum Estimation (ADAM)

ADAM combines momentum with adaptive normalization, using mini-batches for efficiency:

1. First moment (biased toward recent gradients): $m_{t+1} := \beta m_t + (1-\beta)\frac{\partial \mathcal{L}(\phi_t)}{\partial \phi}$
2. Second moment (squared gradients): $v_{t+1} := \gamma v_t + (1-\gamma)\left(\frac{\partial \mathcal{L}(\phi_t)}{\partial \phi}\right)^2$

- $\beta, \gamma \in [0, 1)$ are decay rates (typically $\beta = 0.9, \gamma = 0.999$).

Early estimates of m and v are biased toward zero due to initialization at zero, so corrections are applied:

1. Bias-corrected first moment: $\hat{m}_{t+1} = \frac{m_{t+1}}{1-\beta^{t+1}}$
2. Bias-corrected second moment: $\hat{v}_{t+1} = \frac{v_{t+1}}{1-\gamma^{t+1}}$
   - These account for the exponential decay bias, which is significant early on but negligible later.
3. Final update:

> **ADAM Update**
>
> $$\phi_{t+1} := \phi_t - \alpha \frac{\hat{m}_{t+1}}{\sqrt{\hat{v}_{t+1}} + \epsilon}$$

ADAM achieves faster convergence, greater stability, and robustness to sparse gradients, making it a default choice for many deep learning tasks. (Reference: Simon J.D. Prince, "Understanding Deep Learning", November 2024)

Here's a Python implementation of a simple ADAM update for illustration:

```
import numpy as np

def adam_update(phi_t, grad_t, m_t, v_t, alpha=0.001, beta=0.9, gamma=0
    """
    Single step of ADAM optimization.
    Args:
        phi_t: Current parameters.
        grad_t: Current gradient.
        m_t, v_t: Previous moments (initially 0).
        alpha, beta, gamma, epsilon: Hyperparameters.
        t: Current timestep.
    Returns:
        phi_next, m_next, v_next: Updated values.
    """
    m_next = beta * m_t + (1 - beta) * grad_t
    v_next = gamma * v_t + (1 - gamma) * (grad_t ** 2)
    m_hat = m_next / (1 - beta ** t)
    v_hat = v_next / (1 - gamma ** t)
    phi_next = phi_t - alpha * m_hat / (np.sqrt(v_hat) + epsilon)
```

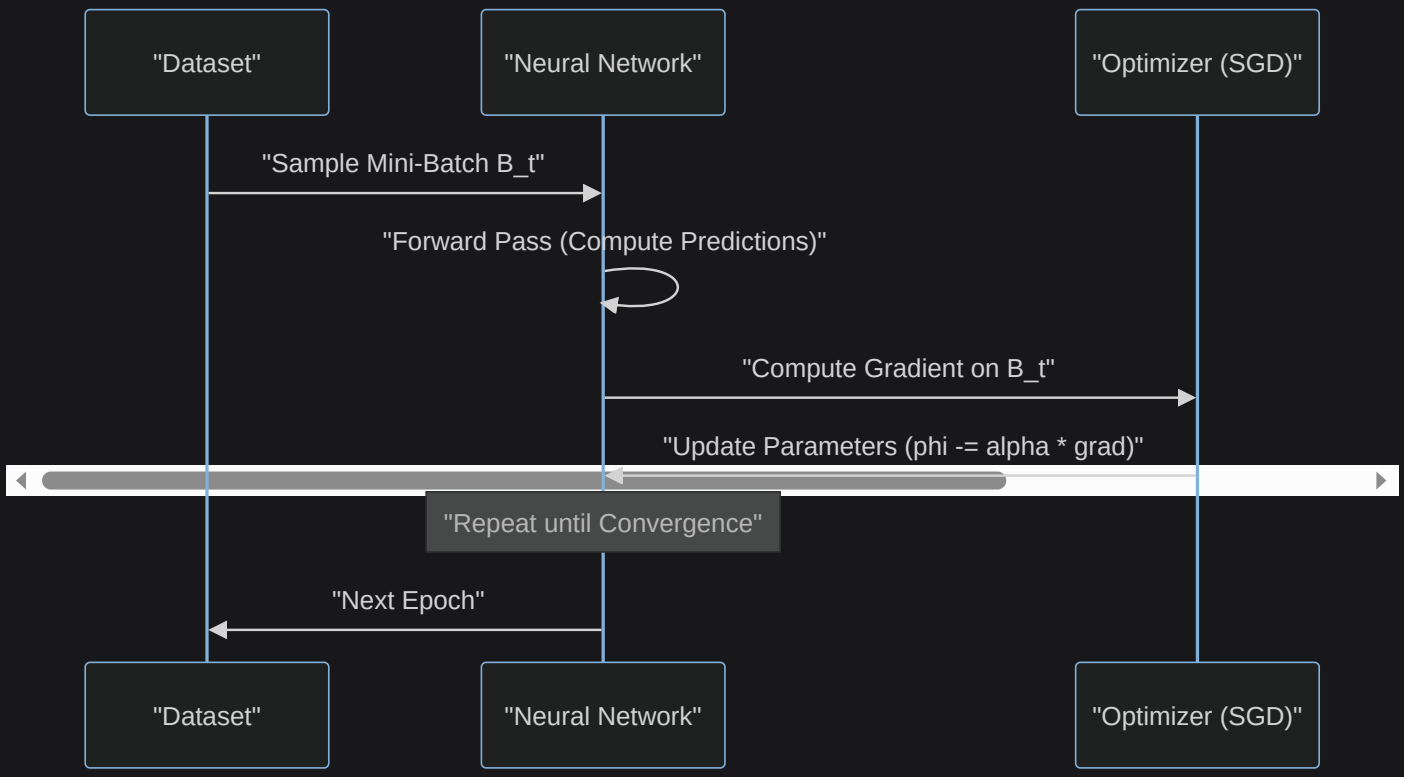
```
    return phi_next, m_next, v_next

# Numerical toy example: 1D, phi_t=2.0, grad_t=-0.5, initial m=0, v=0,
phi_next, m_next, v_next = adam_update(2.0, -0.5, 0, 0, t=1)
print(f"Updated phi: {phi_next:.4f}")  # Example: Moves toward minimum
```

Summary of Optimization

Key choices like the optimization algorithm (e.g., GD, SGD, ADAM), batch size, learning rate α, and momentum coefficient β are **hyperparameters**—settings that influence training dynamics but are not part of the core model parameters or architecture. Selecting optimal hyperparameters is both an art and a science, often involving training multiple configurations and the best performer via **hyperparameter search** (e.g., grid search or random search).

For a visual overview of optimization progression, consider this Mermaid sequence diagram showing the interaction in SGD:



Backpropagation

To compute the gradient $\nabla_\phi \mathcal{L}(\phi)$ efficiently for any loss and model, we use **backpropagation**. This algorithm applies the chain rule in reverse, starting from the end of the computational graph and propagating derivatives backward through the network.

Using the Chain Rule

The chain rule is the mathematical foundation: for a composite function $f(g(x))$, the derivative is $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial g}\frac{\partial g}{\partial x}$. Backpropagation extends this to deep, multi-layered computations by breaking them into sequential operations and computing partial derivatives step by step from output to input.

Computational Graph

A computational graph is a directed acyclic graph where nodes represent operations (e.g., multiplication, addition) and edges represent data flow. It visualizes how inputs combine to produce the output, making differentiation straightforward.

> **Simple Linear Expression Graph**
>
> *Consider computing $y = wx + q$, a simple linear expression.*
>
> - *Variables: Inputs w, x, q.*
> - *Intermediate: $a = wx$ (multiplication node).*
> - *Output: $z = a + q = wx + q$ (addition node).*
>
> *The graph flows from $w, x \to a \to z$, with q joining at the addition.*

Backpropagation Example

Let's apply backpropagation to a toy model with data (x, y), parameters ϕ_1, ϕ_2, and loss $\mathcal{L} = (\phi_1\phi_2 x - y)^2$. The computational graph is:

- $a = \phi_1\phi_2$ (multiplication).
- $b = ax = \phi_1\phi_2 x$ (multiplication).
- $c = b - y = \phi_1\phi_2 x - y$ (subtraction).
- $\mathcal{L} = c^2 = (\phi_1\phi_2 x - y)^2$ (squaring).

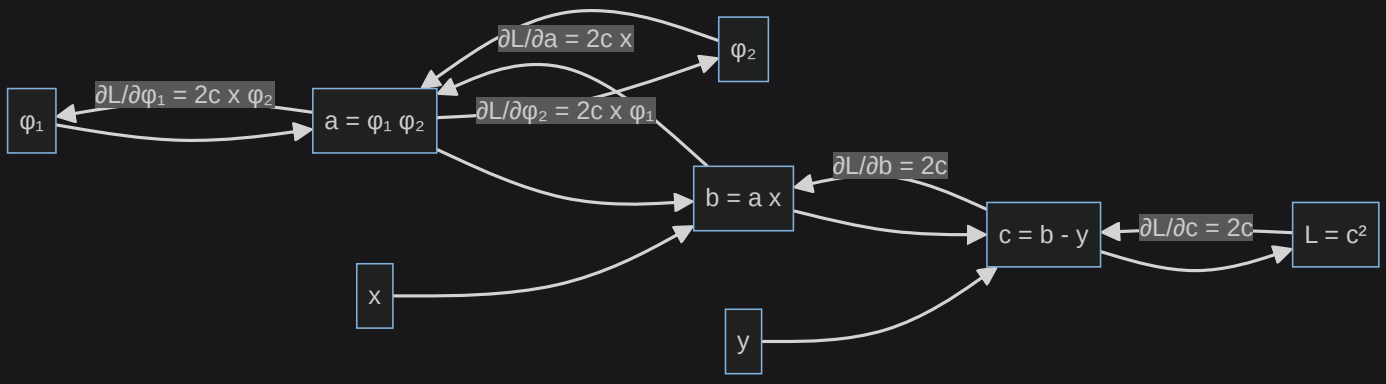
Forward Pass: Starting from inputs ϕ_1, ϕ_2, x, y, compute intermediates a, b, c, and finally $\mathcal{L}$.

Backward Pass: Compute derivatives starting from $\mathcal{L}$ and propagate back:

- $\frac{\partial \mathcal{L}}{\partial c} = 2c$ (derivative of square).
- $\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial c} \cdot \frac{\partial (b-y)}{\partial b} = 2c \cdot 1 = 2c$ (chain through subtraction).
- $\frac{\partial \mathcal{L}}{\partial a} = \frac{\partial \mathcal{L}}{\partial b} \cdot \frac{\partial (ax)}{\partial a} = 2c \cdot x = 2cx$ (chain through multiplication by x).
- $\frac{\partial \mathcal{L}}{\partial \phi_1} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial \phi_1} = 2cx \cdot \phi_2 = 2(\phi_1\phi_2 x - y)x\phi_2$ (chain through multiplication of $\phi_1\phi_2$).
- $\frac{\partial \mathcal{L}}{\partial \phi_2} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial \phi_2} = 2cx \cdot \phi_1 = 2(\phi_1\phi_2 x - y)x\phi_1$ (symmetric for ϕ_2).

These gradients are used to update ϕ_1 and ϕ_2 via an optimizer like GD.

To clarify the backward flow, here's a Mermaid flowchart:



Impact on Loss Functions

The choice of loss function $\mathcal{L}$ directly influences what the model learns, as it defines the optimization objective. Different tasks require tailored losses to penalize errors appropriately.

Regression

For regression, losses measure the discrepancy between predicted $\hat{y}_i$ and true y_i values:

- **MSE** (Mean Squared Error): $\frac{1}{N}\sum(y_i - \hat{y}_i)^2$ – This is simple, differentiable, and penalizes large errors quadratically, making it sensitive to outliers.
- **MAE** (Mean Absolute Error): $\frac{1}{N}\sum|y_i - \hat{y}_i|$ – More robust to outliers, as it uses linear penalties, but less differentiable at zero (subgradients used in practice).

> **Regression Loss Calculation**
>
> *For predicting house prices with $N = 3$ samples: true $[100k, 200k, 150k]$, predicted $[110k, 190k, 160k]$. MSE = $\frac{(10k)^2+(-10k)^2+(10k)^2}{3} \approx 33333k^2$; MAE = $\frac{10k+10k+10k}{3} = 10k$.*

Binary Classification

For binary classification, true labels $y \in \{0, 1\}$ and predictions $\hat{y} = model(x) \in [0, 1]$ (probabilities from sigmoid). The **Binary Cross-Entropy (BCE)** loss is standard:

> **Binary Cross-Entropy**
>
> $$\mathcal{L} = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

This is averaged over the batch. It acts as a selector:

- If $y = 1$ (true positive), it simplifies to $-\log \hat{y}$, penalizing low $\hat{y}$ (confident wrong) heavily while being near-zero for $\hat{y} \approx 1$.
- If $y = 0$ (true negative), it becomes $-\log(1 - \hat{y})$, penalizing high $\hat{y}$ similarly.

BCE is differentiable and commonly used in neural networks for its probabilistic interpretation.

Multi-Class Classification

For multi-class, use **Categorical Cross-Entropy**: $-\sum_{i=1}^{C} y_i \log \hat{y}_i$, averaged over samples.

- y_i: One-hot encoded true labels (1 for correct class, 0 otherwise).
- $\hat{y}_i$: Softmax probabilities.

This encourages the model to assign high probability to the correct class and low to others.

The Importance of the Activation Function

Certain activation functions are unsuitable for deep networks due to the **vanishing gradients** problem, where gradients computed during backpropagation become exponentially small as they propagate backward. This causes early layers to receive negligible updates, stalling learning.

- Examples: Sigmoid and Tanh, being bounded and saturating (gradients near 0 for large |x|), exacerbate this in deep nets.
- Better alternatives: ReLU and its variants (e.g., Leaky ReLU), which have constant gradient (1 for positive inputs), allowing better gradient flow and effective training of deep architectures.

Choosing activations that mitigate vanishing gradients is crucial for scaling to deep networks.

How to Actually Train the Network

Training a neural network is a systematic process that combines data preparation, model design, optimization, and evaluation. Below, we outline the key steps in detail.

Training Steps

1. **Define problem/data**:
   - Clearly specify the objective, such as classification (categorizing data), regression (predicting continuous values), clustering (grouping similar data), anomaly detection (identifying outliers), or generation (creating new data).
   - Preprocess the data: Clean it by removing noise or missing values, augment it (e.g., rotate images for variety), balance classes if imbalanced, and transform features (e.g., normalize to [0,1]).
   - Split the dataset into train (for learning), validation (for tuning), and test (for final evaluation) sets, typically 70/15/15.
   - Choose a batch size based on hardware and dataset size.

2. **Model engineering**: Design the DNN architecture, selecting layers, neurons, activations, and connections to match the problem.

3. **Optimization/loss**: Select an appropriate loss function $\mathcal{L}$ and optimizer (e.g., ADAM), along with hyperparameters like learning rate.

4. **Train/val/optimize**:
   - Perform hyperparameter tuning by experimenting with different values.
   - Evaluate performance on validation data during training.
   - Study overfitting (model memorizes training data) or underfitting (model too simple) by comparing train and validation losses.

Mini-Batch Size

The mini-batch size impacts training time, stability, and generalization. Common choices are powers of 2 (e.g., 32, 64) for hardware efficiency like GPU parallelism.

- **1** (pure stochastic): Processes data as a stream.
  - PRO: Frequent updates, low memory usage.
  - CONS: Very noisy gradients, slower overall convergence.
- **16-64**: Suitable for small datasets or limited hardware.
  - PRO: Balances noise for good generalization on small data.
  - CONS: Still relatively slow due to many updates.
- **128-512**: Standard for medium-sized datasets.
  - PRO: Good trade-off between speed and stability.
  - CONS: Requires more memory.
- **1024+**: For large datasets with ample compute.
  - PRO: Smoother gradients, faster per epoch.
  - CONS: May overfit if batches are too large, reducing generalization.

In practice, start with 32 or 64 and adjust based on validation performance.

Different Problems, Different Loss Functions

The loss $\mathcal{L}$ guides what the optimizer minimizes, so it must align with the task:

- **Regression**:
  - MSE: $\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$ – Emphasizes large errors, smooth optimization.
  - MAE: $\frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$ – Robust to outliers, median-like predictions.
- **Classification**:
  - BCE (Binary Cross-Entropy or Log Loss): $-\frac{1}{N}\sum_{i=1}^{N}[y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)]$

- For binary tasks; $y_i \in \{0, 1\}$, $\hat{y}_i \in [0, 1]$. It's differentiable and maximizes log-likelihood, common in NNs.

- Categorical Cross-Entropy: $-\sum_{i=1}^{C} y_i \log \hat{y}_i$ (averaged over N)
  - For multi-class; y_i one-hot, $\hat{y}_i$ from softmax. Penalizes confident wrong predictions heavily.

Performance Evaluation Metrics

While loss functions drive training, evaluation metrics provide intuitive, task-specific measures of success on held-out data (validation/test sets).

Classification:

- **Accuracy**: $\frac{\text{Number of Correct Predictions}}{\text{Total Predictions}}$ – Simple overall correctness, but misleading on imbalanced datasets (e.g., 99% negative class inflates accuracy).

For each class C_i, more nuanced metrics include:

- **Precision(C_i)**: $\frac{\text{True Positives for } C_i}{\text{True Positives for } C_i + \text{False Positives for } C_i}$ – Fraction of C_i that are actually C_i (minimizes false alarms).
- **Recall(C_i)**: $\frac{\text{True Positives for } C_i}{\text{True Positives for } C_i + \text{False Negatives for } C_i}$ – Fraction of actual C_i correctly identified (minimizes misses).
- **F1(C_i)**: $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$ – Harmonic mean, balancing precision and recall for imbalanced classes.

> **Spam Detection Metrics**
>
> *In a binary classifier for spam emails (positive=spam), with 100 predictions: 20 true spam, 80 non-spam. If model predicts 25 spam (22 true, 3 false): Precision=22/25=0.88, Recall=22/20=1.10 (wait, cap at 1; adjust example), F1 balances them.*

For regression, metrics like MSE/MAE (as losses) or R^2 (explained variance) are used.

Underfitting and Overfitting

Monitor both training and validation losses over epochs to detect fitting issues:

- **Underfitting**: The model is too simple (e.g., few neurons/layers) and fails to capture patterns. Both train and val losses remain high.
- **Overfitting**: The model memorizes training data but generalizes poorly. Train loss decreases steadily (often to near 0), while val loss initially drops but then rises.

(Reference: Abhishek Shrivastava, “Underfitting Vs Just right Vs Overfitting in Machine learning”, Kaggle)

A typical loss curve plot shows: Underfitting (high plateau), Just Right (both converge low), Overfitting (train low, val rises).

Address Underfitting

To resolve underfitting and increase model capacity:

- **Preprocess better**: Remove noise/outliers, encode categorical features properly.
- **Enhance model**: Add more neurons/layers, change architecture (e.g., deeper MLP), increase epochs, or reduce batch size for more updates.

Address Overfitting

Regularization techniques constrain the model to improve generalization, especially vital for high-capacity neural networks:

- **Weights**:
  - **L1 (Lasso)**: Add penalty $\lambda \sum_{i=0}^{n} |\phi_i|$ (sum of absolute weights). Promotes sparsity by driving some weights to exactly zero, aiding feature selection. $\lambda > 0$ controls strength.
  - **L2 (Ridge)**: Add $\lambda \sum_{i=0}^{n} \phi_i^2$ (sum of squared weights). Shrinks weights toward zero without sparsifying, distributing importance evenly.
  - Combined loss: $\mathcal{L} = \mathcal{L}_{std} + \text{penalty}$, where $\mathcal{L}_{std}$ is the standard loss (e.g., MSE).
- **Architecture**:
  - **Dropout**: During training, randomly set a fraction (e.g., 0.5) of neurons to zero (visualized as grayed out). This prevents co-adaptation of features, making the network robust. At inference, all neurons are used, but activations are scaled.
- **Training**:
  - **Early Stopping**: Monitor validation loss; halt training if it degrades for several epochs, preventing overfitting.
- **Data**:
  - **Batch Normalization**: Normalize inputs to each layer (mean 0, variance 1) during training. Stabilizes and accelerates training, reduces sensitivity to initialization, and mildly regularizes to curb overfitting. Can be applied before or after activations.
  - **Noise Injection**: Add random noise to inputs, weights, or activations (e.g., Gaussian noise). Builds robustness by simulating real-world variations.

Validation loss plots often show the progression from underfitting to overfitting; stopping at the optimal point is key. Including batch normalization layers further smooths these curves.

References

- Machine Learning
- Neural Networks
- Linear Algebra

- Cybenko, George. "Approximation by superpositions of a sigmoidal function." Mathematics of control, signals and systems 2.4 (1989): 303-314.
- Hornik, Kurt, Maxwell Stinchcombe, and Halbert White. "Multilayer feedforward networks are universal approximators." Neural networks 2, no. 5 (1989): 359-366.
- Simon J.D. Prince, “Understanding Deep Learning”, November 2024.
- Abhishek Shrivastava, “Underfitting Vs Just right Vs Overfitting in Machine learning”, Kaggle.